4b

CloudWatch Monitoring y Observabilidad

Curso: AWS Cloud Practitioner Essentials

4 horas

Objetivos de Aprendizaje

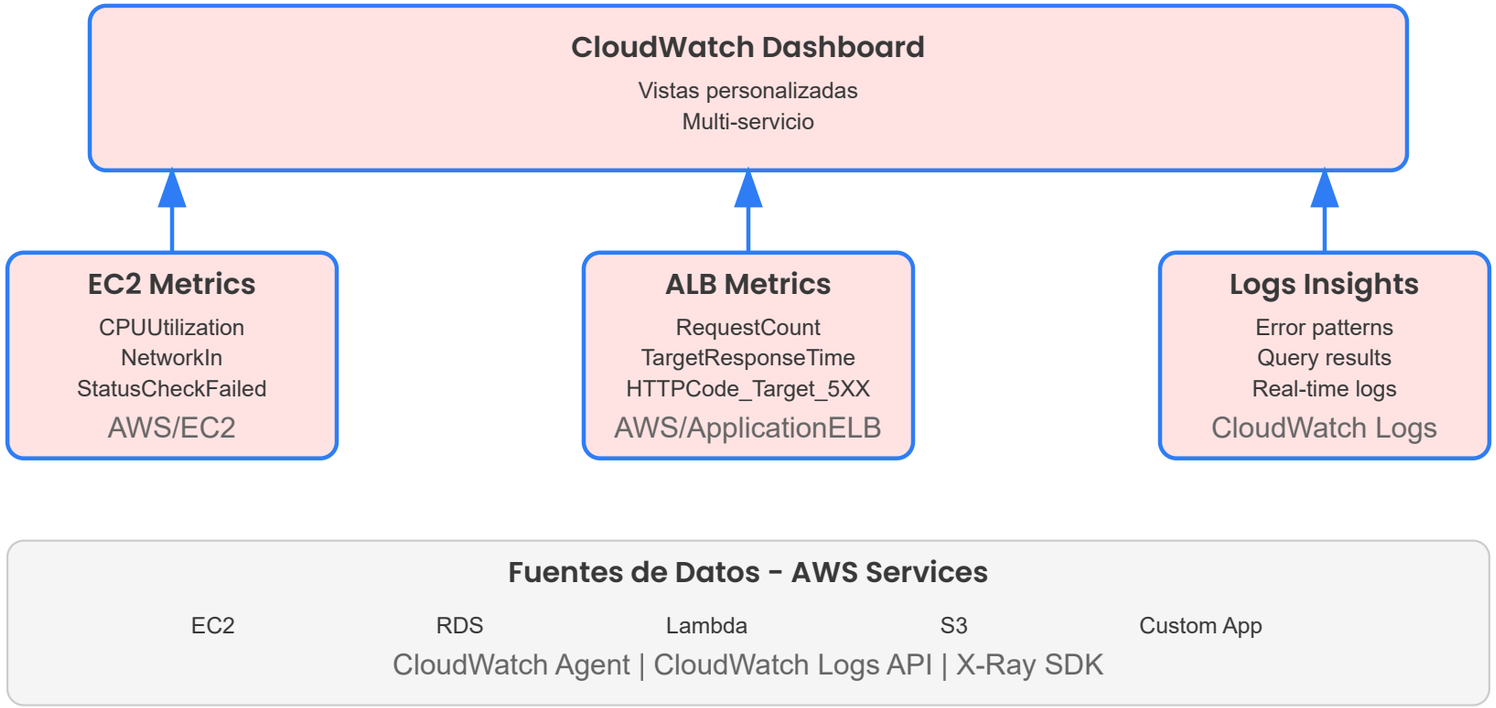
- Crear y personalizar dashboards de CloudWatch para monitoreo en tiempo real de múltiples servicios
- Utilizar CloudWatch Logs Insights para análisis avanzado de logs con queries estructuradas
- Implementar CloudWatch Contributor Insights para identificar contributors top-N y patrones de errores
- Configurar CloudWatch Application Insights para detección automática de problemas en aplicaciones
- Integrar métricas de múltiples servicios en vistas unificadas con ServiceLens

1. CloudWatch Dashboards

Monitoreo en tiempo real multi-servicio

Arquitectura de Dashboards

Arquitectura CloudWatch Dashboards



¿Qué es un CloudWatch Dashboard?

Definición: Vista personalizada que permite monitorear recursos de AWS en tiempo real

Los dashboards pueden incluir:

- **Métricas graficadas:** Visualización de métricas de cualquier servicio AWS
- **Widgets de texto:** Para agregar contexto y anotaciones
- **Widgets de alarma:** Para mostrar estados de CloudWatch Alarms
- **Widgets de logs:** Para mostrar patrones de errores detectados

Tipos de Widgets Disponibles

Tipo	Descripción	Caso de Uso
`metric`	Gráfico de métricas estándar	Monitorear CPU, memoria
`log`	Query de Logs Insights embebida	Mostrar errores en tiempo real
`text`	Texto estático o markdown	Anotaciones y contexto
`alarm`	Estado de alarma específico	Alertas activas
`custom`	Widget personalizado con JSON	Integraciones especiales

APIs de Dashboard - AWS CLI

```
# Crear dashboard
aws cloudwatch put-dashboard \
    --dashboard-name "MiDashboard" \
    --dashboard-body '{
        "widgets": [
            {
                "type": "metric",
                "properties": {
                    "metrics": [["AWS/EC2", "CPUUtilization", "InstanceId", "i-1234567890abcdef0"]],
                    "period": 300,
                    "stat": "Average",
                    "region": "us-east-1",
                    "title": "CPU EC2"
                }
            }
        ]
    }'

# Listar dashboards
```

Métricas Compuestas

Definición: Expresiones matemáticas sobre múltiples métricas

```
{
    "metrics": [
        ["AWS/EC2", "CPUUtilization", "InstanceId", "i-1234567890abcdef0"],
        [".", "NetworkIn", ".", "."],
        [{ "expression": "m1/m2*100", "label": "Ratio" }]
    ]
}
```

Lab 4b.1: Dashboard Personalizado

30 minutos

Objetivo del Lab

Crear un dashboard que monitoree servicios de aplicación web en tiempo real: EC2, ALB y CloudWatch Logs

Prerrequisitos

- Instancia EC2 con agente CloudWatch instalado
- Application Load Balancer configurado
- Logs de aplicación enviando a CloudWatch Logs

Pasos del Lab

1. Acceder a CloudWatch > Dashboards > Create dashboard
2. Agregar widget de métricas EC2 (CPUUtilization)
3. Agregar widget de métricas ALB (TargetResponseTime, p95)
4. Configurar widget de Logs Insights para errores
5. Agregar widget de alarma CPU-High-Alarm
6. Organizar layout y guardar dashboard

Verificación del Lab

Verificación	Estado
Dashboard visible en consola CloudWatch	[ ]
Widget de EC2 muestra datos de CPU en tiempo real	[ ]
Widget de ALB muestra latencia	[ ]
Widget de logs muestra errores recientes	[ ]
Alarma visible si existe violación de umbral	[ ]

2. CloudWatch Logs Insights

Análisis avanzado de logs con queries estructuradas

Lenguaje de Query

CloudWatch Logs Insights - Lenguaje de Query

Estructura de Query

```
FIELDS @timestamp, @message, @level
| FILTER @message LIKE /error/
| PARSE @message '* - - * * * * * * *' as ip, method, path
| STATS avg(latency) as avgLat by path
```

FIELDS

Extraer campos
Crear campos calculados
Renombrar

@timestamp | @message
@ingestionTime | @log
@logStream

FILTER

LIKE /pattern/
= "exact_value"
AND | OR

LIKE /(?i)error/
Case insensitive

PARSE

Extraer patrones
Delimitados
JSON automatico

parse @message
'* * * * * *' as...

STATS

count(*)
avg | min | max | sum
percentile(c, p)

by bin(1h) as hour
Agrupacion temporal

Ejemplo: Errores por Hora

```
fields @timestamp, @message
| filter @message like /(?i)error/
| stats count(*) as errors by bin(1h) as hour
| sort hour desc
```

Ejemplo: Parseo de Logs ACCESS_LOG

```
parse @message '* - - * * * * * * *' as ip, method, path, status, size, latency, protocol
```

Sintaxis Básica

```
FIELDS @timestamp, @message, @ingestionTime
| FILTER @message LIKE /error/
| SORT @timestamp DESC
| LIMIT 20
```

Componentes del Lenguaje

FIELDS

- Extraer campos específicos
- Crear campos calculados
- Renombrar

FILTER

- LIKE /pattern/
- = "exact_value"
- AND | OR

STATS y PARSE

STATS

- count(*), avg, min, max, sum
- percentile(campo, p)
- Agrupación temporal: bin(1h)

PARSE

- Extraer patrones
- Logs delimitados
- JSON automático

Funciones de Agregación

Función	Descripción	Ejemplo
`count(*)`	Cuenta registros	stats count(*) as total
`avg(campo)`	Promedio	stats avg(latencyMs)
`min(campo)`	Valor mínimo	stats min(duration)
`max(campo)`	Valor máximo	stats max(cpu)
`percentile(campo, p)`	Percentil	stats percentile(latency, 95)

Ejemplos de Queries Comunes

```
-- Errores por hora
fields @timestamp, @message
| filter @message like /(?i)error/
| stats count(*) as errors by bin(1h) as hour
| sort hour desc

-- Top 10 IPs con más errores
fields remoteAddr, @message
| filter @message like /error/
| stats count(*) as errorCount by remoteAddr
| sort errorCount desc
| limit 10
```

Parsing de Logs Estructurados

```
-- Logs con formato: TIMESTAMP - METHOD PATH STATUS LATENCYms
fields @message
| parse @message '* - * * * *' as logTimestamp, method, path, status, latency
| filter path like /\/api\//
| stats avg(latency) as avgLatency by path
| sort avgLatency desc

-- Logs JSON estructurados
fields @message
| parse @message '{ "level": "*", "msg": "*", "service": "*" }' as level, msg, service
| filter level = "error"
```

Lab 4b.2: Análisis de Logs

40 minutos

Escenario

Una aplicación web está experimentando tiempos de respuesta lentos. El equipo necesita:

1. Identificar los endpoints más lentos
2. Determinar la causa de errores frecuentes
3. Crear un dashboard para monitoreo continuo

Pasos del Lab

1. Explorar los logs disponibles
2. Identificar errores 5xx
3. Calcular latencia promedio por endpoint
4. Crear visualización de errores por hora
5. Crear dashboard con queries guardadas
6. Configurar alarma para errores

Comandos AWS CLI para Logs Insights

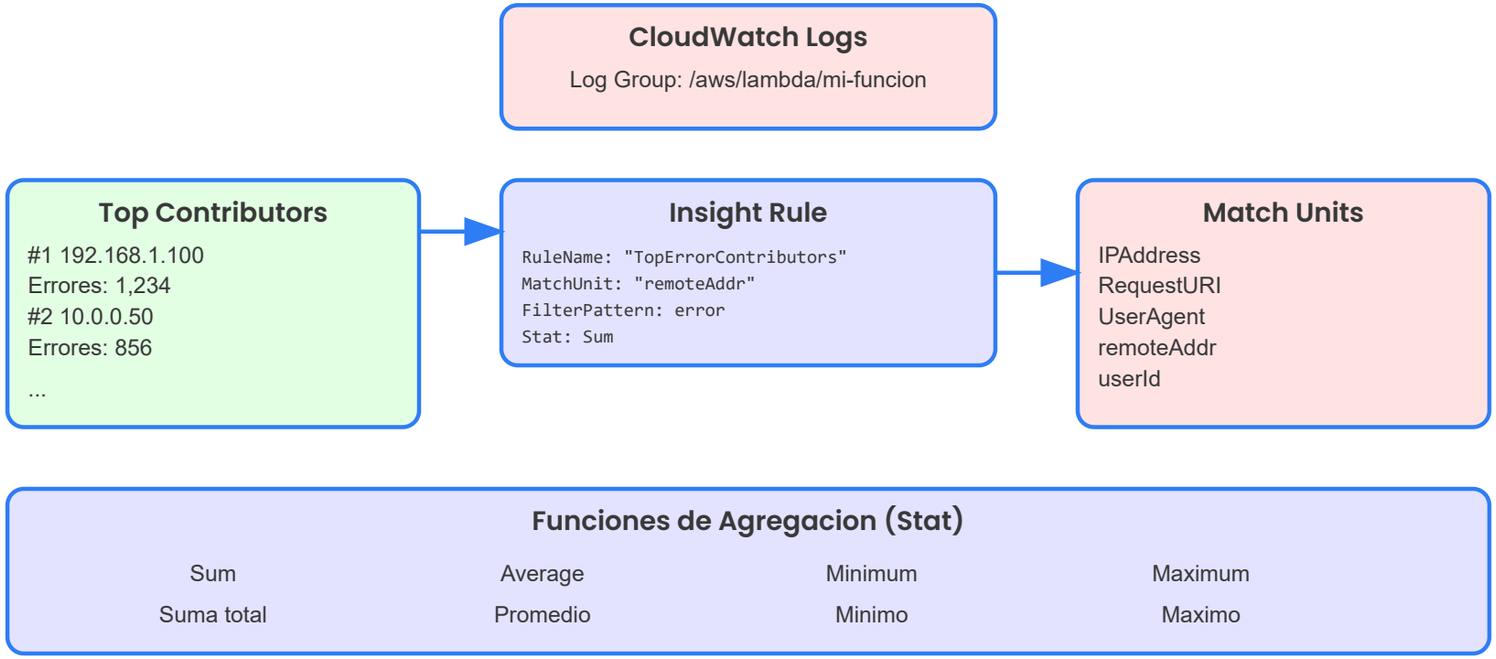
```
# Ejecutar query directamente
aws logs start-query \
    --log-group-name "/aws/lambda/mi-funcion" \
    --start-time 1713600000 \
    --end-time 1713603600 \
    --query-string 'fields @timestamp, @message | filter @message like /error/ | limit 20'

# Obtener resultados (usar queryId retornado)
aws logs get-query-results \
    --query-id "<query-id>"
```

3. CloudWatch Contributor Insights

Identificación de contributors top-N

Arquitectura de Contributor Insights

CloudWatch Contributor Insights - Arquitectura



¿Qué es Contributor Insights?

Analiza logs para identificar los principales contributors a métricas específicas

Casos de uso:

- Encontrar las IPs que generan más solicitudes
- Identificar usuarios con más errores
- Descubrir endpoints más utilizados

Tipos de Match Units

Match Unit	Descripción	Ejemplo de Uso
`IPAddress`	Dirección IP	Top IPs causando errores
`RequestURI`	URI de request	Endpoints más llamados
`UserAgent`	User agent string	Clientes más comunes
`remoteAddr`	IP del cliente	Fuentes de tráfico
`userId`	ID de usuario	Usuarios más activos

Crear regla via AWS CLI

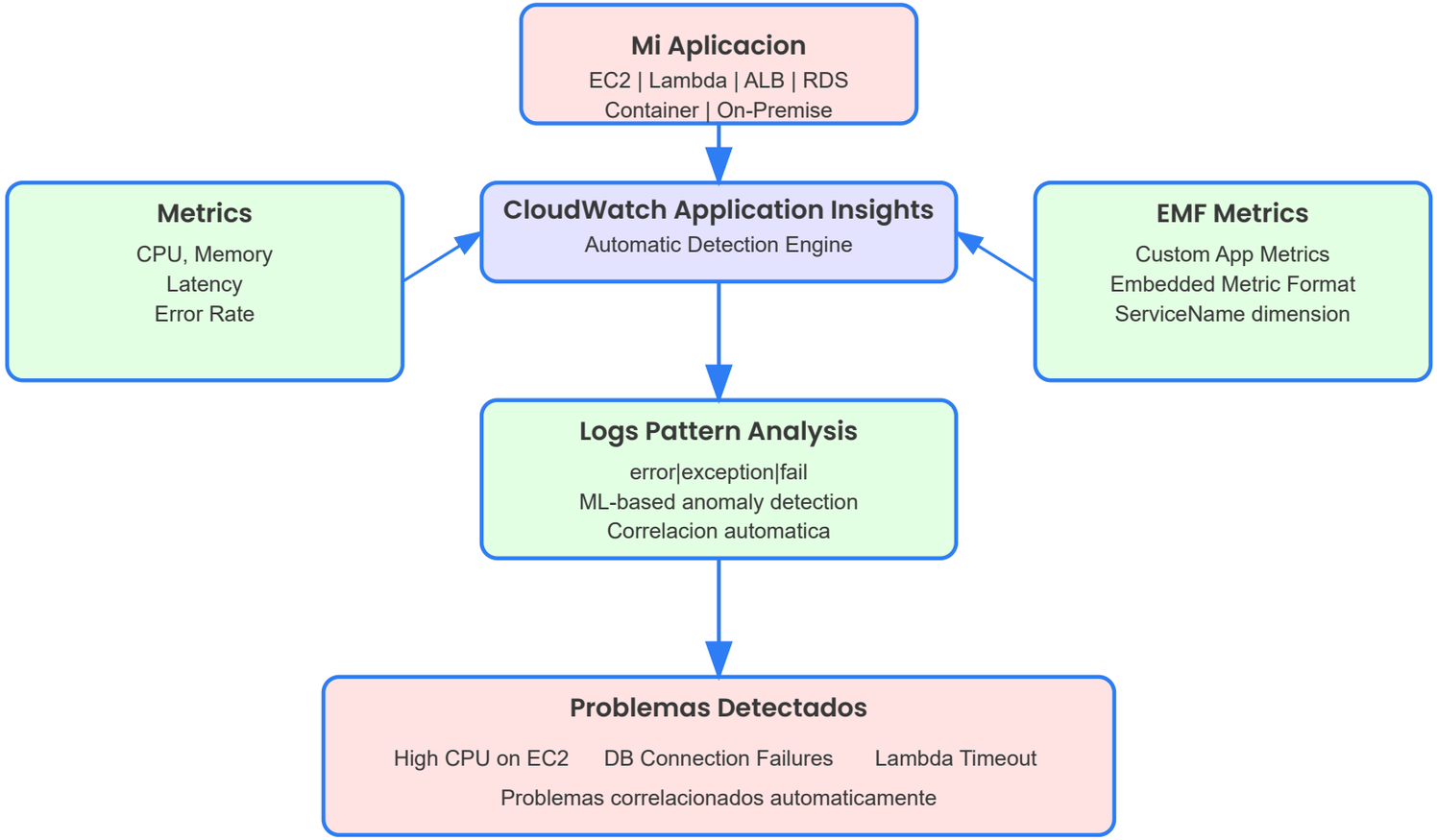
```
aws cloudwatch put-insight-rule \
    --rule-name "TopErrorContributors" \
    --rule-state ENABLED \
    --rule-definition '{
        "Schema": {"Name": "CloudWatchLogRule", "Version": 1},
        "AggregateOn": "Count",
        "Contribution": {
            "Keys": ["$.remoteAddr"],
            "Filters": [{"Match": "$.status", "GreaterThan": 499}]
        },
        "LogFormat": "JSON",
        "LogGroupNames": ["/aws/lambda/mi-funcion"]
    }'
```

4. CloudWatch Application Insights

Detección automática de problemas

Arquitectura de Application Insights

CloudWatch Application Insights - Detección Automática



¿Qué es Application Insights?

Monitoreo automatizado para aplicaciones. Detecta problemas basándose en:

- Métricas de infraestructura
- Logs de aplicación
- Correlación de errores

Características Principales

- **Detección automática de anomalías:** Usa machine learning para identificar comportamientos anómalos
- **Correlación de problemas:** Agrupa symptoms relacionados
- **EMF (Embedded Metric Format):** Permite a aplicaciones enviar métricas personalizadas

Formato EMF

```
{
    "_aws": {
        "Timestamp": "2026-04-20T10:30:00.000Z",
        "CloudWatchMetrics": [{
            "Namespace": "MyApplication",
            "Dimensions": [["ServiceName"]],
            "Metrics": [{
                "Name": "RequestLatency",
                "Unit": "Milliseconds"
            }]
        }]
    },
    "ServiceName": "checkout-service",
    "RequestLatency": 145
}
```

Configuración

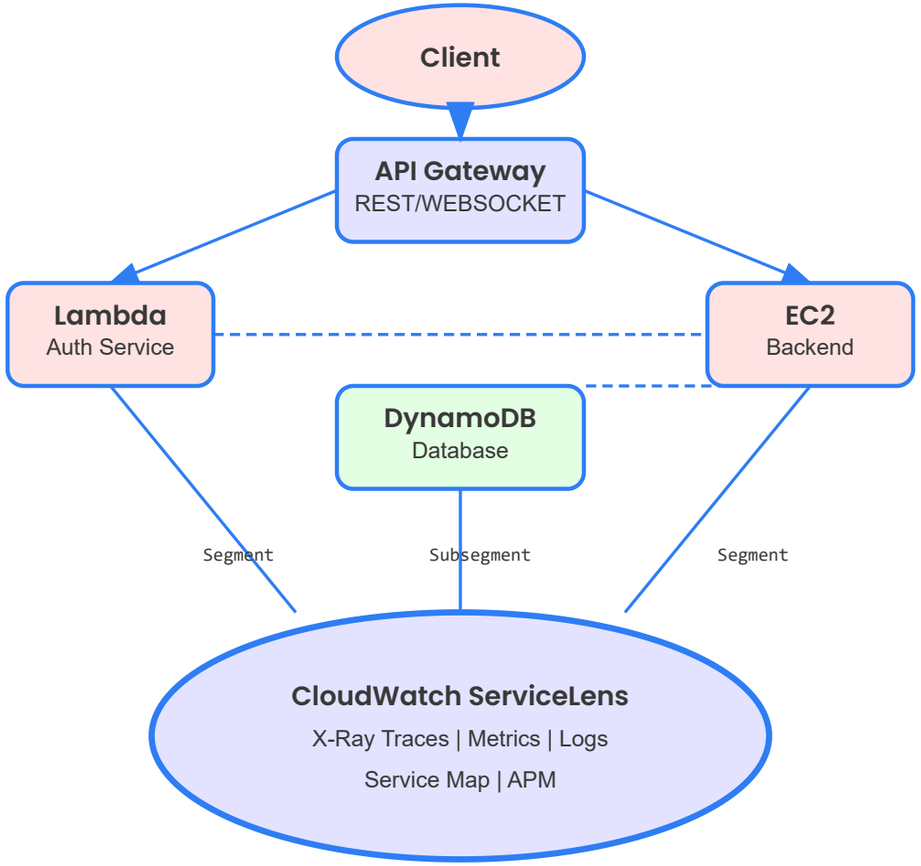
1. Ir a CloudWatch > Application Insights > Create
2. Seleccionar grupo de recursos
3. Especificar componentes de aplicación
4. Habilitar detección automática

5. CloudWatch ServiceLens

Traces y service maps - Observabilidad unificada

Arquitectura ServiceLens

CloudWatch ServiceLens - Arquitectura de Observabilidad



Trace: Request completo Segment: Datos de un servicio Subsegment: Componentes internos Service Map: Dependencias

Introducción a ServiceLens

ServiceLens combina:

- **X-Ray traces:** Información de requests distribuidos
- **CloudWatch Metrics:** Métricas de servicios
- **CloudWatch Logs:** Logs de aplicación
- **Service Maps:** Visualización de dependencias

Conceptos de X-Ray

Concepto	Descripción
Trace	Request completo a través de múltiples servicios
Segment	Datos de un solo servicio
Subsegment	Componentes internos de un servicio

Configurar X-Ray en Lambda

```
# Habilitar tracing en Lambda
aws lambda update-function-configuration \
    --function-name mi-funcion \
    --tracing-config Mode=Active

# Ver traces en Console
# CloudWatch > ServiceLens > X-Ray traces
```

Analizar un Trace

1. Ir a CloudWatch > ServiceLens > Traces
2. Filtrar por tiempo y servicio
3. Click en un trace para ver detalles
4. Identificar latency en cada segmento

Lab 4b.3: Application Insights

30 minutos

Objetivo del Lab

Configurar CloudWatch Application Insights para una aplicación con EC2, Lambda y ALB. El monitoreo automatizado detectará problemas sin configuración manual de umbrales.

Prerrequisitos

- Resource Group `ProductionApp-RG` con EC2, Lambda y ALB
- CloudWatch Agent instalado en EC2
- Logs de aplicación en CloudWatch Logs
- Permisos IAM: `applicationinsights:*`, `resource-groups:*`

Pasos del Lab

1. CloudWatch > Application Insights > Create application
2. Seleccionar Resource Group: `ProductionApp-RG`
3. Verificar componentes detectados (EC2, Lambda, ALB)
4. Configurar patrones de log (`error|exception|fail|timeout`)
5. Verificar métricas habilitadas por componente
6. Revisar pestaña Problems (esperar 5-10 min)

Verificación del Lab

Verificación	Estado
Application Insight con estado "Monitoring"	[ ]
EC2, Lambda y ALB visibles en Components	[ ]
3 patrones de log configurados	[ ]
Al menos 3 métricas por componente habilitadas	[ ]
Pestaña Problems accesible	[ ]

Lab 4b.4: Tracing con ServiceLens y X-Ray

45 minutos

Objetivo del Lab

Implementar tracing distribuido con X-Ray y ServiceLens para identificar latencia en una aplicación serverless con Lambda, API Gateway, DynamoDB y SQS.

Prerrequisitos

- Funciones Lambda: `lambda-auth`, `lambda-orders`, `lambda-notification`
- API Gateway con integración Lambda
- Tabla DynamoDB y cola SQS
- Policy `AWSXRayDaemonWriteAccess` en rol de Lambda

Pasos del Lab

1. Habilitar Active Tracing en las 3 funciones Lambda
2. Verificar via CLI: `TracingConfig.Mode=Active`
3. CloudWatch > ServiceLens > Service Map (esperar 5 min)
4. Identificar nodos y conexiones del service map
5. Seleccionar trace con Duration > 2000ms y analizar
6. Correlacionar trace con logs via "View logs"

Verificación del Lab

Verificación	Estado
Tracing Active en 3 funciones Lambda	[ ]
Service Map muestra ≥ 4 nodos	[ ]
Conexiones reflejan arquitectura real	[ ]
Trace seleccionable con subsegmentos	[ ]
Logs filtrables por Trace ID	[ ]

Resumen

Puntos Clave

- **CloudWatch Dashboards** permiten crear vistas personalizadas de métricas y logs en tiempo real
- **Logs Insights** proporciona un lenguaje de query poderoso para análisis de logs
- **Contributor Insights** identifica los principales contributors a problemas
- **Application Insights** detecta problemas automáticamente usando ML
- **ServiceLens** unifica traces, métricas y logs en una sola vista

Comandos AWS CLI Clave

```
# Dashboards
aws cloudwatch list-dashboards
aws cloudwatch put-dashboard --dashboard-name X --dashboard-body Y
aws cloudwatch delete-dashboards --dashboard-names X

# Logs Insights
aws logs start-query --log-group-name X --start-time T1 --end-time T2 --query-string Y
aws logs get-query-results --query-id Z

# Contributor Insights
aws cloudwatch put-insight-rule --rule-name X --rule-state ENABLED --rule-definition Y
aws cloudwatch list-insight-rules

# Application Insights
aws applicationinsights create-application --resource-group-name X
```

Mejores Prácticas

- Usar dashboards para monitoreo proactivo, no solo reactivo
- Guardar queries comunes en Logs Insights para reutilización
- Configurar Contributor Insights para identificar fuentes de problemas
- Habilitar Application Insights en todas las aplicaciones de producción
- Integrar ServiceLens para debugging distribuido

Referencias

Documentación Oficial AWS

- CloudWatch Dashboards
- CloudWatch Logs Insights
- CloudWatch Contributor Insights
- CloudWatch Application Insights
- CloudWatch ServiceLens

Workshops Complementarios

- AWS CloudWatch Workshop
- X-Ray Trace Workshop

Gracias!

4b Completado

Siguiente: Módulos de práctica integrada